

NNGraph DSL Compiler

A Domain-Specific Language for Describing Neural Network Architectures
and Automatic PyTorch Code Generation

Project Technical Report

July 2, 2026

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 Goals	4
1.3 Scope	5
2 Related Work	6
2.1 The NADER Framework	6
3 Tools and Tech Stack	7
3.1 ANTLR4	7
3.2 Python	7
3.3 PyTorch	7
4 Architecture Overview	8
5 Language Design	9
5.1 Program Structure	9
5.2 Grammar	10
5.3 Type System	11
5.4 Layer Types	11
5.4.1 Parameterized Layers	11
5.4.2 Special Operations	12
5.5 Edge Syntax	13
5.5.1 Comments	13
6 Implementation	14
6.1 Compiler Architecture	14
6.2 Parsing	14
6.3 Semantic Analyzer	16
6.4 Code Generator	18

6.5	Shape Inference	27
6.6	Graphviz Visualization	28
6.7	ONNX Export	30
7	Conclusion and Future Work	31
7.1	Conclusion	31
7.2	Future Work	31
	References	32

Abstract

NNGraph DSL is a domain-specific language designed to describe neural network architectures as directed graphs. It accepts **.nng** files as input and produces executable PyTorch code consisting of a complete **nn.Module** subclass with **__init__** and **forward** methods (plus possibly a training loop). The compiler is built with ANTLR4 version 4.13.2.

Tensor shape inference, Graphviz graph visualization, training scaffolding, quantization annotations, and ONNX export are also supported.

Introduction

1.1 Motivation

Writing `nn.Module` PyTorch codes by hand (or directly via LLM response) is repetitive, error-prone, and most importantly somehow unnecessary! So this project aims to automate and simplify this process by declaring a simple DSL called NNGraph so users can write this simpler syntax (or ask their AI tools to output in this syntax) instead of the more complex PyTorch syntax. This can be helpful for several reasons:

1. Writing this syntax is easier for both humans and AI agents.
2. This syntax is less error-prone as the compiler provides useful error checkings as well.
3. AI agents can use less tokens if paired with this tool like this: the agent will output code in `.nng` format then uses this tool to convert it into PyTorch code to present to user; this makes the agent less error-prone (can catch its errors earlier) and also makes it use less tokens!
4. Because of the error checkings and utilities this compiler provides, programmers can also diagnose what's wrong easier and earlier.
5. This approach is memory optimized as it allocates the minimum amount of variables needed that can be annoying and often forgotten to be handled by humans (and LLM agents).
6. This compiler provides easier generation of boilerplate codes, a good example is Quantization that we will examine later.

1.2 Goals

- Express any feed-forward or branching neural network topology as a graph.
- Generate clean, idiomatic PyTorch code.
- Support common layer types, activations, normalization, and skip connections.

- Be implementation-agnostic — the DSL is the single source of truth.
- Allow inline hyperparameter configuration per node.

Note that the generated code is meant to be a starting point and it's clear that code can be further expanded as needed.

1.3 Scope

The implementation supports code generation (`nn.Module`, `__init__`, `forward()`, training loop and setups) plus basic semantic checks and also shape inference checks, debug and visualization tools, ONNX export and quantization annotations.

But it does not support dynamic control flow meaning the graph is static and known at model definition time.

We also do not have checks for loss function to match the architecture.

Macros and dynamic control flow is planned as future work.

2.1 The NADER Framework

Definition 2.1.1 ▶ NADER

NADER (Neural Architecture Design via Representation) is a framework that models neural network architectures as directed computational graphs (DAGs). Each node represents a layer operation and each edge represents tensor flow.

NNGraph DSL adopts this graph model as the foundation of its language design.

Tools and Tech Stack

3.1 ANTLR4

ANTLR4 (ANother Tool for Language Recognition) is a tool that automatically generates a lexer, parser, and listener/visitor from a **.g4** grammar file. Version used: ANTLR 4.13.2.

This was the main tool used to implement the compiler.

3.2 Python

Python 3.14.3

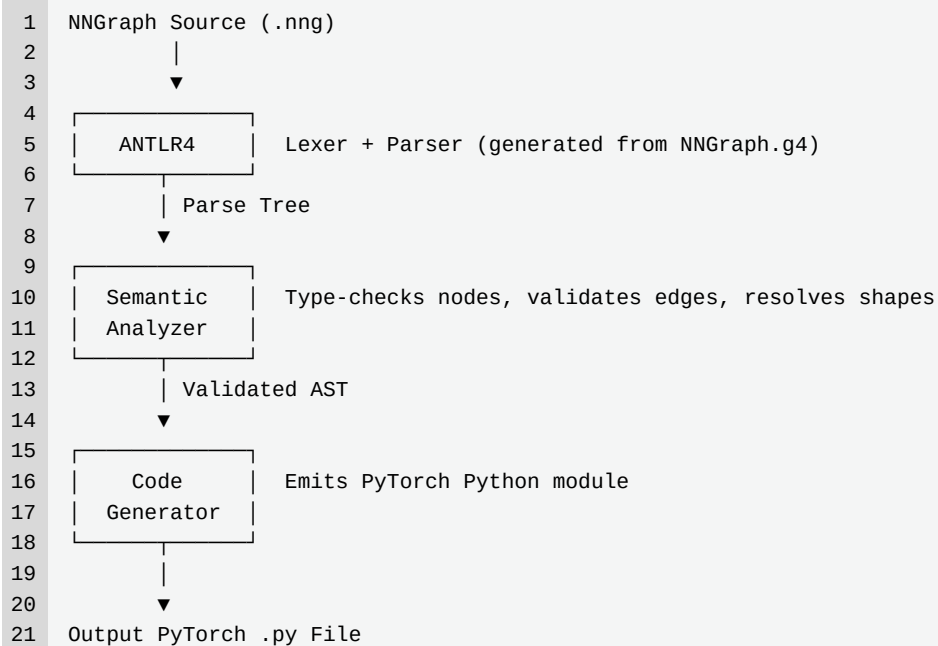
3.3 PyTorch

We didn't directly use PyTorch in this project but as we are generating code the version 2.10.0 was used for code generation and correctness verification.

Architecture Overview

The toolchain follows a classic compiler pipeline:

Listing 4.0.1 ▶ Architecture Pipeline



Language Design

5.1 Program Structure

Every `.nng` file consists of three blocks. The `config` block is optional:

Listing 5.1.1 ▶ General structure of a `.nng` file

```

1  model <Name> {
2      input  <id> : tensor(<shape>)
3      output <id>
4  }
5
6  graph {
7      node <id> : <LayerType>(<params>)
8      ...
9      edge <src> -> <dst>
10     edge <src> -> <dst> [label="<string>"]
11     ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
17     loss        = "<LossFn>"           // enables training
18     ↪ scaffold
19     optimizer    = "<Optimizer>"       // Adam | SGD | AdamW |
20     ↪ RMSprop
21     lr           = <float>              // learning rate
22     epochs       = <int>                // training epochs
23 }
```

Definition 5.1.2 ▶ Model block

The **model** block specifies the generated class name, the input node identifier and tensor shape, and the output node identifier.

Definition 5.1.3 ▶ Graph block

The **graph** block defines all layer nodes and directed edges. Edge declaration order does not matter; the compiler computes execution order from the graph structure.

Definition 5.1.4 ▶ Config block

The **config** block sets values for the `__main__` block of the generated code. **batch_size** defaults to 1 and **device** defaults to "cpu". If the **loss** key is set, the code generator produces a complete training loop with optimizer and loss function.

5.2 Grammar

The grammar is simple and short, complete grammar rules:

Listing 5.2.1 ▶ NNGraph.g4 grammar

```

1  grammar NNGraph;
2  start      : model_block graph_block config_block? EOF;
3  model_block : MODEL ID '{' input_decl output_decl '}';
4  input_decl  : INPUT ID ':' TENSOR shape_expr;
5  output_decl : OUTPUT ID;
6  graph_block : GRAPH '{' (node_decl | edge_decl)* '}';
7  node_decl   : NODE ID ':' layer_expr;
8  layer_expr  : ID '(' param_list? ')';
9  param_list  : param (',' param)*;
10 param       : ID '=' value;
11 edge_decl   : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block : CONFIG '{' config_entry* '}';
13 config_entry : ID '=' value;
14 value        : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15              | STRING | NONE | shape_expr;

```

```
16 shape_expr : '(' INT_LITERAL (',' INT_LITERAL)* ')';
```

5.3 Type System

DSL Type	Example	Description
<code>int</code>	<code>128</code>	Integer literal
<code>float</code>	<code>0.1</code>	Floating-point literal
<code>bool</code>	<code>true / false</code>	Boolean flag
<code>string</code>	<code>"relu"</code>	Quoted string
<code>shape</code>	<code>(3, 224, 224)</code>	Tuple of ints for tensor shape
<code>None</code>	<code>None</code>	Optional absent value

Note: The distinction between `int` and `float` is enforced at compile time. For example, `Dropout(p=1)` produces an error because `p` expects a `float`, not an `int`.

5.4 Layer Types

5.4.1 Parameterized Layers

DSL Keyword	PyTorch Mapping	Key Parameters
Linear	<code>nn.Linear</code>	<code>in_features</code> , <code>out_features</code> , <code>bias</code>
Conv2d	<code>nn.Conv2d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code> , <code>padding</code>
Conv1d	<code>nn.Conv1d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>kernel</code> , <code>stride</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>kernel</code> , <code>stride</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>start_dim</code> , <code>end_dim</code>
Embedding	<code>nn.Embedding</code>	<code>num_embeddings</code> , <code>embedding_dim</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>embed_dim</code> , <code>num_heads</code>
LSTM	<code>nn.LSTM</code>	<code>input_size</code> , <code>hidden_size</code> , <code>num_layers</code>
GRU	<code>nn.GRU</code>	<code>input_size</code> , <code>hidden_size</code>

5.4.2 Special Operations

Keyword	Parameters	Generated Code
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a,b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int</code> , <code>dim: int</code>	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>

5.5 Edge Syntax

Definition 5.5.1 ▶ Simple edge

```
edge src -> dst
```

Defines a directed edge from node **src** to node **dst**.

Definition 5.5.2 ▶ Labeled edge

```
edge src -> dst [label="name"]
```

Defines an edge with a string label. Labels serve two roles:

- **"shortcut"**: marks the skip connection edge into a **Residual()** node.
- **"residual_***": marks a skip connection into a node with two inputs (Transformer pattern).

Note that labels are used for these purposes only in this implementation to improve clarity.

5.5.1 Comments

Listing 5.5.3 ▶ Comments syntax

```
1 // This is a single-line comment
2
3 /* This is a
4    multi-line comment */
```

Implementation

6.1 Compiler Architecture

Definition 6.1.1 ▶ Compilation pipeline

The compiler has six stages:

1. Parsing: `.nng` file $\rightarrow$ `NNGraphLexer` $\rightarrow$ `NNGraphParser.program()` $\rightarrow$ parse tree
2. Semantic analysis: `ParseTreeWalker` + `NNGraphCustomListener` $\rightarrow$ graph model (+ AST - which is not used apart from debugging)
3. Topological sort: compute execution order using Kahn's algorithm
4. Shape inference: propagate tensor shapes through the graph and detect dimension mismatches (optional: `--show-shapes`)
5. Code generation: `CodeGenerator.generate()` $\rightarrow$ Python string
6. Output: write to `.py` file (+ optional: Graphviz DOT output, or ONNX export)

6.2 Parsing

This part is done automatically by ANTLR4. Here are some parse tree visualizations based on our grammar `.g4` file from the ANTLR extension in PyCharm environment (see Figures 6.1, 6.2, and 6.3):

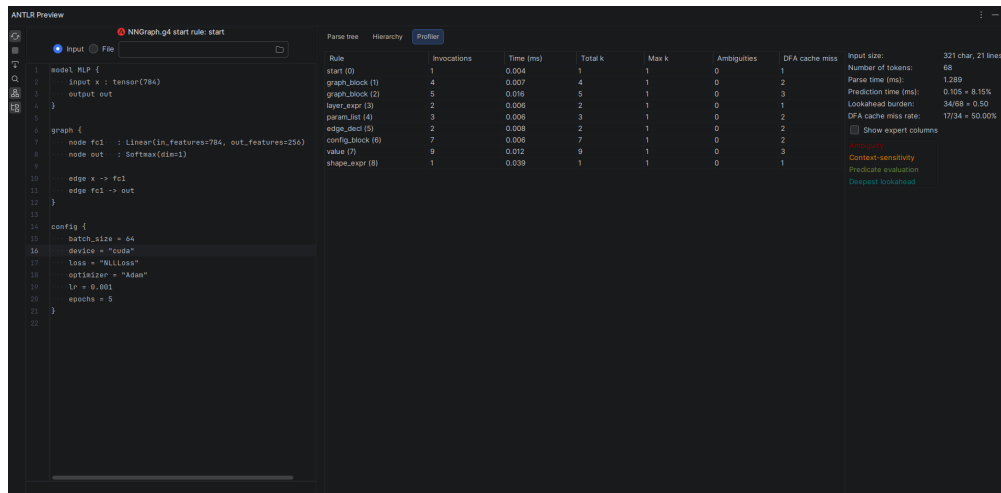


Figure 6.1: Parse Tree Visualization 1

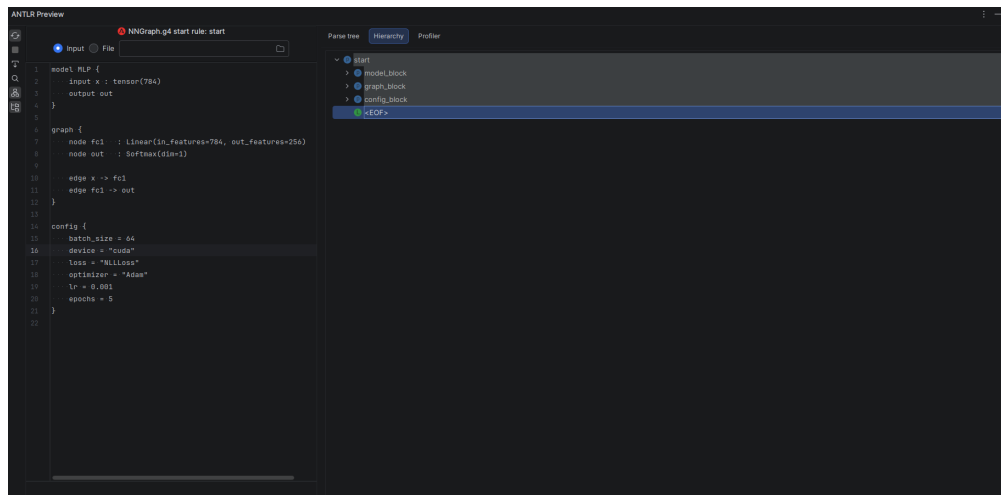


Figure 6.2: Parse Tree Visualization 2

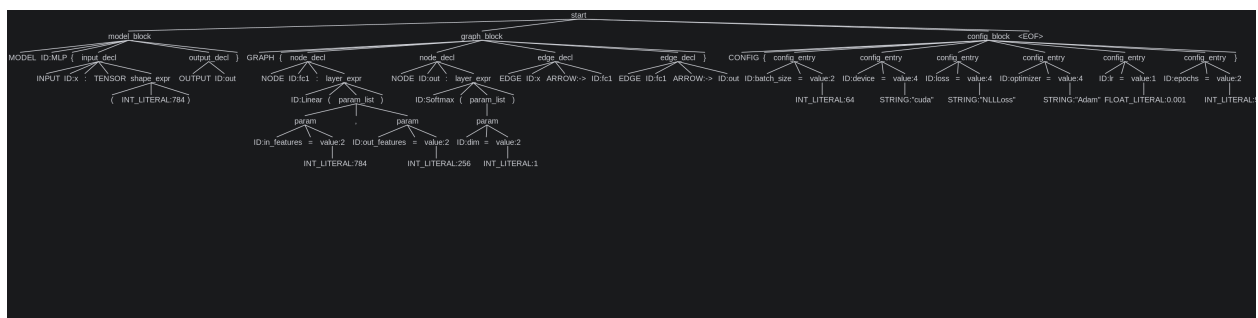


Figure 6.3: Detailed Parse Tree

6.3 Semantic Analyzer

We use a **CustomListener** approach to analyze the code semantically. This works like this: we declare functions to be executed once we exit (wanna move on from traversing) each node using the tools provided for us from ANTLR4.

3 kinds of work is done:

1. Populate **ctx** (context - ANTLR provides a **ctx** for each node with some info - we use those and add some more fields based on our needs) fields for each node (using its children nodes or nodes that were traversed before in general) - and global lists and dictionaries - these will be used in kind 2 when doing the semantic checks (these will be used in Code Generation too and can be considered the output of Semantic Analysis Step).
2. Do Semantic Checks and output errors if needed using the info from kind 1.
3. AST Creation: AST will be created by **exitEveryRule** calling the **make_ast_subtree** on all nodes which creates the AST subtree, since AST is not crucial to this project we will not examine the details further.

Kind 1 is routine and therefore we skip explaining it in more detail.

The validations done in kind 2 are these:

1. **Check for duplicate node id:** In **exitNode_decl** we check to confirm the node id is not already in nodes dictionary.
2. **Check for unknown layer type:** Check to confirm the layer type is declared in our fixed **LAYER_SCHEMA** in **exitNode_decl**.
3. **Check for correct parameter types:** Using **LAYER_SCHEMA** and **UNIVERSAL_PARAMS** (quant) also in **exitNode_decl** (this checks if each layer has correct parameters and the parameter values have correct type).
4. **Check for correct quant value:** Check to confirm the value of quant param is correct using **QUANT_MODES** also in **exitNode_decl**.
5. **Check if the nodes exist in edge declaration:** In **exitEdge_decl**.
6. **Check for output declaration:** After the graph block is completed → we simple check if output is in nodes.

7. **Check for orphan nodes:** For each edge we make a referenced flag for both src and dst true and see if any node has a false flag after.
8. **Check if residual nodes have exactly 2 inputs:** We find residuals and check if they have 2 `in_edges`.
9. **Cycle detection:** We use Kahn's algorithm to determine if the graph is a DAG: in each step we remove the node with no `in_edges` and then update the `in_edges` for other nodes until the graph has no more nodes, if in any step there is no node with 0 `in_edges` the graph has a cycle.
10. **Reachability:** We use BFS to traverse the graph and check if all nodes are reachable from input (including output).

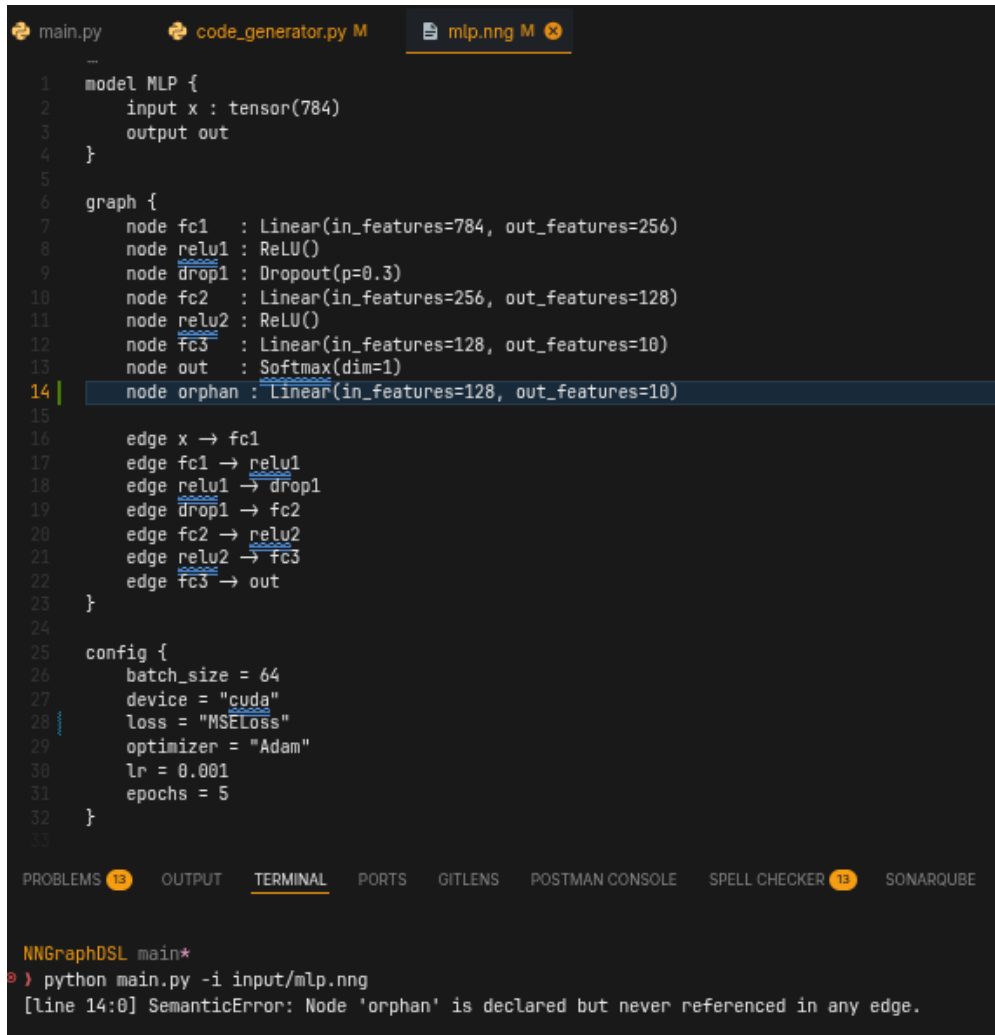
Note: items 6 to 10 are done in `exitGraph_block` after the graph block is walked and nodes and edges are filled.

If in any case those validations fail we will have an error in this format:

Listing 6.3.1 ▶ Error format

```
1 [line L:C] SemanticError: <message>
```

Example:



```

main.py  code_generator.py M  mlp.nng M X
...
1  model MLP {
2      input x : tensor(784)
3      output out
4  }
5
6  graph {
7      node fc1 : Linear(in_features=784, out_features=256)
8      node relu1 : ReLU()
9      node drop1 : Dropout(p=0.3)
10     node fc2 : Linear(in_features=256, out_features=128)
11     node relu2 : ReLU()
12     node fc3 : Linear(in_features=128, out_features=10)
13     node out : Softmax(dim=1)
14 |  node orphan : Linear(in_features=128, out_features=10)
15
16     edge x → fc1
17     edge fc1 → relu1
18     edge relu1 → drop1
19     edge drop1 → fc2
20     edge fc2 → relu2
21     edge relu2 → fc3
22     edge fc3 → out
23 }
24
25 config {
26     batch_size = 64
27     device = "cuda"
28     loss = "MSELoss"
29     optimizer = "Adam"
30     lr = 0.001
31     epochs = 5
32 }
33
NNGraphDSL main*
python main.py -i input/mlp.nng
[line 14:0] SemanticError: Node 'orphan' is declared but never referenced in any edge.

```

Figure 6.4: Semantic Error Example

6.4 Code Generator

Code generation is done via the info gathered by the last step which is Semantic Analysis (like nodes and edges and `model_name` and ...).

It has 6 steps:

1. `_compute_graph`: Uses the same Kahn's algorithm but this time for topological sort, we need topological sort because it's essential to first process the input nodes of a node then that node to be able to generate the code correctly, the algorithm is simple too just like before but after removing a node we append it to the `topo_order` too.
2. `_assign_vars`: This assigns variables to each node but uses 4 rules to ensure mini-

memory allocation along with correct behavior:

- (a) For input node always allocate **x**.
 - (b) For merge nodes (**in_degree** > 1): if the out degree is 1 or 0 allocate **x** - if out degree is more than 1 allocate a variable named as **node_id**.
 - (c) Branch nodes (predecessor **out_degree** > 1): allocate a variable named as **node_id** (split nodes' out nodes also get named similarly but with a small difference: the variable is named like this: **source_name** + **'_'** + **index of the node**).
 - (d) And if **out_degree** is 1 we will allocate the same variable allocated to its predecessor to reuse memory.
3. **_emit_init**: For this part we need to get PyTorch class name for each node that has a module and also make its params string in correct format. We have a map to map each node (not in nodes with no modules like Add) to its PyTorch class name and also get the names of params that it accepts. Then with those param names we use **_build_params** to build the params string like this: if the value has type string wrap it in double quotes if not just simply set it like **param_name = param_value** and another exception is for **MultiHeadAttn** layer, we always add this param to it: **batch_first=True**. Now we have the data we need, so for each node we first check if quantization is enabled, if yes we wrap the code into a PyTorch Quantization wrapper if not we emit the normal code, and in the end if any of the nodes had quantization enabled we emit code for Stubs too. In this step topo order does not matter but we use topo order anyway.
4. **_emit_forward**: This uses the helper **_emit_node** to emit the line for each node. First we examine the behavior for special nodes:

Residual: We determine the shortcut edge (which is the edge that has the shortcut label) and also the main branch edge (which does not have shortcut label). Then we get the related vars to the src of those edges and generate the code like: **out_var** (determined by getting **var_at[current_node]**) = **main_var** + **shortcut_var**.

Add: For Add we do a similar job: we add all input vars and assign back to the **out_var**.

Concat: This is similar to Add but we also dim from the params of that node and use **torch.cat** instead of simple +.

Split: This is again similar to Concat but we need to get chunks and dim from params and use `torch.chunk`.

Now for nodes that have Modules: One special case is implicit Residuals when the node has more than 1 input. We handle these like normal Residuals with the difference that we look for the edges that their label start with `residual` to mark as shortcut and we give the addition of those `main_var` and `shortcut_var` to the module input. If no such edges with residual label found then we fall-back to default behavior where we simply use the first edge's src's var as the input of the module.

Normal case is when we have one input, but there is 2 special nodes in that case too: **MultiHeadAttn** which has a special line of code and also **LSTM** and **GRU** which have another special line. If not in these types the normal line is emitted.

At the end back in `_emit_forward` we check and if any node had quant param in the very beginning we add a `x = self.quant(x)` line and at the end use dequant like this:

Listing 6.4.1 ▶ Dequantize output

```
1 {self.var_at[self.output_id]} = self.dequant({self.var_at[self.output_id]})
```

5. **_assemble:** This first generates the import and class declaration boilerplate then adds init lines and forward lines (from the last 2 steps) and in the end calls `_emit_training` to optionally generate training loop.
6. **_emit_training:** We first check and if loss function is not specified we will return empty (no training loop). If not we simply map the grammar values (like optimizer, lr and epochs) to PyTorch syntax and place them in the straightforward training loop boilerplate.

MLP Input (`input/mlp.nng`):

Listing 6.4.2 ▶ MLP Input

```
1 model MLP {
2     input x : tensor(784)
3     output out
4 }
5
6 graph {
7     node fc1 : Linear(in_features=784, out_features=256)
8     node relu1 : ReLU()
```

```

9      node drop1 : Dropout(p=0.3)
10     node fc2   : Linear(in_features=256, out_features=128)
11     node relu2 : ReLU()
12     node fc3   : Linear(in_features=128, out_features=10)
13     node out   : Softmax(dim=1)
14
15     edge x -> fc1
16     edge fc1 -> relu1
17     edge relu1 -> drop1
18     edge drop1 -> fc2
19     edge fc2 -> relu2
20     edge relu2 -> fc3
21     edge fc3 -> out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27     loss = "MSELoss"
28     optimizer = "Adam"
29     lr = 0.001
30     epochs = 5
31 }

```

MLP Output (`output/mlp.py`):

Listing 6.4.3 ▶ MLP Output

```

1  import torch
2  import torch.nn as nn
3
4  class MLP(nn.Module):
5      def __init__(self):
6          super(MLP, self).__init__()
7          self.fc1 = nn.Linear(in_features=784, out_features=256)
8          self.relu1 = nn.ReLU()
9          self.drop1 = nn.Dropout(p=0.3)
10         self.fc2 = nn.Linear(in_features=256, out_features=128)
11         self.relu2 = nn.ReLU()
12         self.fc3 = nn.Linear(in_features=128, out_features=10)
13         self.out = nn.Softmax(dim=1)
14
15     def forward(self, x):
16         x = self.fc1(x)
17         x = self.relu1(x)
18         x = self.drop1(x)
19         x = self.fc2(x)
20         x = self.relu2(x)
21         x = self.fc3(x)
22         x = self.out(x)
23         return x
24
25 if __name__ == '__main__':

```

```

26     device = torch.device('cuda')
27     model = MLP().to(device)
28     x = torch.randn(64, 784).to(device)
29     print(model(x).shape)
30
31     criterion = nn.MSELoss()
32     optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
33
34     # Training loop
35     model.train()
36     for epoch in range(5):
37         x = torch.randn(64, 784).to(device)
38         y = model(x)
39         target = torch.zeros_like(y)
40         loss = criterion(y, target)
41         optimizer.zero_grad()
42         loss.backward()
43         optimizer.step()
44         print(f'Epoch {epoch+1}/5, Loss: {loss.item():.4f}')

```

ResBlock Input (`input/resblock.nng`):

Listing 6.4.4 ▶ ResBlock Input

```

1  model ResBlock {
2      input x : tensor(64, 32, 32)
3      output out
4  }
5
6  graph {
7      node conv1 : Conv2d(in_ch=64, out_ch=64, kernel=3, stride=1, padding=1)
8      node bn1   : BatchNorm2d(num_features=64)
9      node relu1 : ReLU()
10     node conv2 : Conv2d(in_ch=64, out_ch=64, kernel=3, stride=1, padding=1)
11     node bn2   : BatchNorm2d(num_features=64)
12     node skip  : Residual()
13     node relu2 : ReLU()
14     node out   : Flatten()
15
16     edge x -> conv1
17     edge conv1 -> bn1
18     edge bn1 -> relu1
19     edge relu1 -> conv2
20     edge conv2 -> bn2
21     edge bn2 -> skip
22     edge x -> skip [label="shortcut"]
23     edge skip -> relu2
24     edge relu2 -> out
25 }
26
27 config {
28     batch_size = 32
29     device = "cpu"

```

```

30     loss = "MSELoss"
31     optimizer = "AdamW"
32     lr = 0.0005
33     epochs = 3
34 }

```

ResBlock Output (`output/resblock.py`):

Listing 6.4.5 ▶ ResBlock Output

```

1  import torch
2  import torch.nn as nn
3
4  class ResBlock(nn.Module):
5      def __init__(self):
6          super(ResBlock, self).__init__()
7          self.conv1 = nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3, stride=1,
8                                  ↪ padding=1)
9          self.bn1 = nn.BatchNorm2d(num_features=64)
10         self.relu1 = nn.ReLU()
11         self.conv2 = nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3, stride=1,
12                                 ↪ padding=1)
13         self.bn2 = nn.BatchNorm2d(num_features=64)
14         self.relu2 = nn.ReLU()
15         self.out = nn.Flatten()
16
17     def forward(self, x):
18         conv1 = self.conv1(x)
19         conv1 = self.bn1(conv1)
20         conv1 = self.relu1(conv1)
21         conv1 = self.conv2(conv1)
22         conv1 = self.bn2(conv1)
23         x = conv1 + x
24         x = self.relu2(x)
25         x = self.out(x)
26         return x
27
28 if __name__ == '__main__':
29     device = torch.device('cpu')
30     model = ResBlock().to(device)
31     x = torch.randn(32, 64, 32, 32).to(device)
32     print(model(x).shape)
33
34     criterion = nn.MSELoss()
35     optimizer = torch.optim.AdamW(model.parameters(), lr=0.0005)
36
37     # Training loop
38     model.train()
39     for epoch in range(3):
40         x = torch.randn(32, 64, 32, 32).to(device)
41         y = model(x)
42         target = torch.zeros_like(y)
43         loss = criterion(y, target)

```



```

42     optimizer.zero_grad()
43     loss.backward()
44     optimizer.step()
45     print(f'Epoch {epoch+1}/3, Loss: {loss.item():.4f}')

```

QuantMLP Input (`input/quant_mlp.nng`):

Listing 6.4.6 ▶ QuantMLP Input

```

1  model QuantMLP {
2      input x : tensor(784)
3      output out
4  }
5
6  graph {
7      node fc1    : Linear(in_features=784, out_features=128, quant="qat")
8      node relu1  : ReLU()
9      node fc2    : Linear(in_features=128, out_features=10, quant="qat")
10     node out    : Softmax(dim=1)
11
12     edge x -> fc1
13     edge fc1 -> relu1
14     edge relu1 -> fc2
15     edge fc2 -> out
16 }
17
18 config {
19     batch_size = 32
20     device = "cpu"
21     loss = "NLLLoss"
22     optimizer = "Adam"
23     lr = 0.001
24     epochs = 2
25 }

```

QuantMLP Output (`output/quant_mlp.py`):

Listing 6.4.7 ▶ QuantMLP Output

```

1  import torch
2  import torch.nn as nn
3
4  class QuantMLP(nn.Module):
5      def __init__(self):
6          super(QuantMLP, self).__init__()
7          self.fc1 = torch.quantization.QuantWrapper(nn.Linear(in_features=784,
8              ↪ out_features=128))
9          self.relu1 = nn.ReLU()
10         self.fc2 = torch.quantization.QuantWrapper(nn.Linear(in_features=128,
11             ↪ out_features=10))
12         self.out = nn.Softmax(dim=1)

```

```

11     self.quant = torch.quantization.QuantStub()
12     self.dequant = torch.quantization.DeQuantStub()
13
14     def forward(self, x):
15         x = self.quant(x)
16         x = self.fc1(x)
17         x = self.relu1(x)
18         x = self.fc2(x)
19         x = self.out(x)
20         x = self.dequant(x)
21         return x
22
23 if __name__ == '__main__':
24     device = torch.device('cpu')
25     model = QuantMLP().to(device)
26     x = torch.randn(32, 784).to(device)
27     print(model(x).shape)
28
29     criterion = nn.NLLLoss()
30     optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
31
32     # Training loop
33     model.train()
34     for epoch in range(2):
35         x = torch.randn(32, 784).to(device)
36         y = model(x)
37         target = torch.zeros_like(y)
38         loss = criterion(y, target)
39         optimizer.zero_grad()
40         loss.backward()
41         optimizer.step()
42         print(f'Epoch {epoch+1}/2, Loss: {loss.item():.4f}')

```

TransformerEncoder Input (`input/transformer.nng`):

Listing 6.4.8 ▶ TransformerEncoder Input

```

1  model TransformerEncoder {
2      input x : tensor(512, 128)
3      output out
4  }
5
6  graph {
7      node attn  : MultiHeadAttn(embed_dim=128, num_heads=8)
8      node drop1 : Dropout(p=0.1)
9      node norm1 : LayerNorm(normalized_shape=128)
10
11     node ff1   : Linear(in_features=128, out_features=512)
12     node gelu1 : GELU()
13     node drop2 : Dropout(p=0.1)
14     node ff2   : Linear(in_features=512, out_features=128)
15     node norm2 : LayerNorm(normalized_shape=128)
16     node out   : Flatten(start_dim=0, end_dim=1)

```

```

17
18     edge x -> attn
19     edge attn -> drop1
20     edge drop1 -> norm1
21     edge x -> norm1 [label="residual_1"]
22
23     edge norm1 -> ff1
24     edge ff1 -> gelu1
25     edge gelu1 -> drop2
26     edge drop2 -> ff2
27     edge ff2 -> norm2
28     edge norm1 -> norm2 [label="residual_2"]
29     edge norm2 -> out
30 }

```

TransformerEncoder Output (`output/transformer.py`):

Listing 6.4.9 ▶ TransformerEncoder Output

```

1  import torch
2  import torch.nn as nn
3
4  class TransformerEncoder(nn.Module):
5      def __init__(self):
6          super(TransformerEncoder, self).__init__()
7          self.attn = nn.MultiheadAttention(embed_dim=128, num_heads=8, batch_first=True)
8          self.drop1 = nn.Dropout(p=0.1)
9          self.norm1 = nn.LayerNorm(normalized_shape=128)
10         self.ff1 = nn.Linear(in_features=128, out_features=512)
11         self.gelu1 = nn.GELU()
12         self.drop2 = nn.Dropout(p=0.1)
13         self.ff2 = nn.Linear(in_features=512, out_features=128)
14         self.norm2 = nn.LayerNorm(normalized_shape=128)
15         self.out = nn.Flatten(start_dim=0, end_dim=1)
16
17     def forward(self, x):
18         attn, _ = self.attn(x, x, x)
19         attn = self.drop1(attn)
20         norm1 = self.norm1(x + attn)
21         ff1 = self.ff1(norm1)
22         ff1 = self.gelu1(ff1)
23         ff1 = self.drop2(ff1)
24         ff1 = self.ff2(ff1)
25         x = self.norm2(norm1 + ff1)
26         x = self.out(x)
27         return x
28
29 if __name__ == '__main__':
30     device = torch.device('cpu')
31     model = TransformerEncoder().to(device)
32     x = torch.randn(1, 512, 128).to(device)
33     print(model(x).shape)

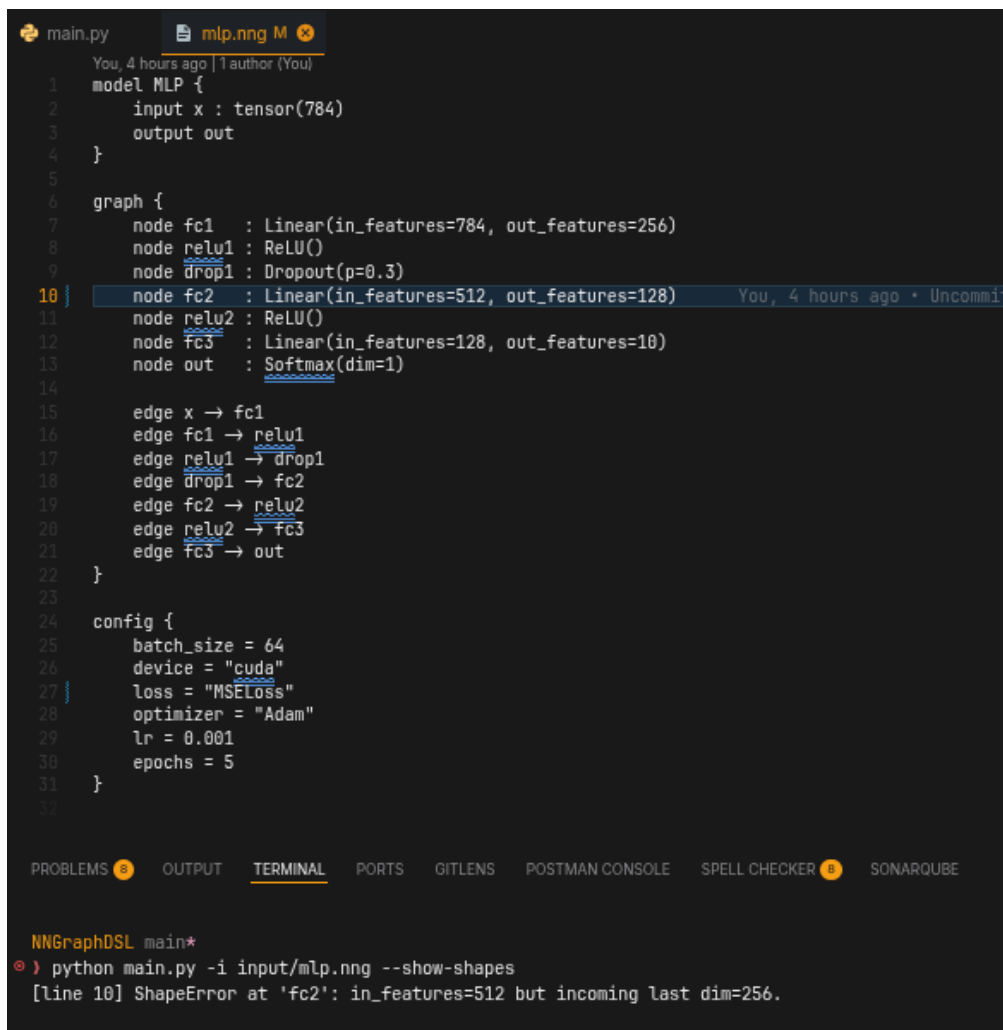
```

6.5 Shape Inference

Definition 6.5.1 ▶ Shape Inference

The `shape_inference.py` module propagates tensor shapes from the input node along the topological order. For each layer type, specific rules are applied: Linear changes the last dimension, Conv2d computes spatial dimensions, Flatten merges dimensions, and so on.

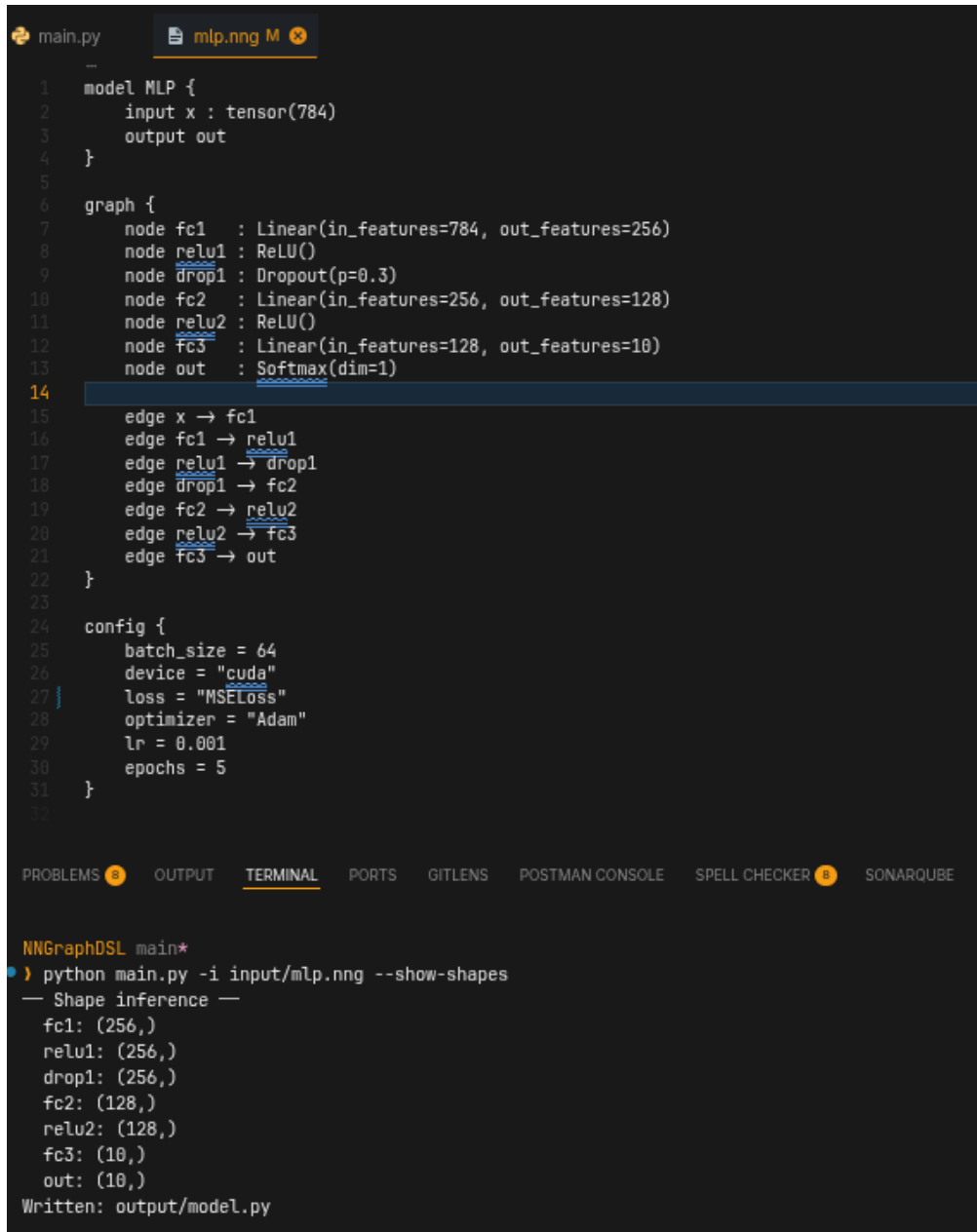
When a dimension mismatch is detected, a `ShapeError` is reported with the line number of the node declaration, and compilation is halted (see Figure 6.5). The `--show-shapes` flag displays each node's output shape (see Figure 6.6).



```

main.py  mlp.nng M
You, 4 hours ago | 1 author (You)
1  model MLP {
2      input x : tensor(784)
3      output out
4  }
5
6  graph {
7      node fc1 : Linear(in_features=784, out_features=256)
8      node relu1 : ReLU()
9      node drop1 : Dropout(p=0.3)
10     node fc2 : Linear(in_features=512, out_features=128)
11     node relu2 : ReLU()
12     node fc3 : Linear(in_features=128, out_features=10)
13     node out : Softmax(dim=1)
14
15     edge x → fc1
16     edge fc1 → relu1
17     edge relu1 → drop1
18     edge drop1 → fc2
19     edge fc2 → relu2
20     edge relu2 → fc3
21     edge fc3 → out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27     loss = "MSELoss"
28     optimizer = "Adam"
29     lr = 0.001
30     epochs = 5
31 }
32
NNGraphDSL main*
python main.py -i input/mlp.nng --show-shapes
[line 10] ShapeError at 'fc2': in_features=512 but incoming last dim=256.
  
```

Figure 6.5: Shape Error Example



```

main.py  mlp.nng M
...
1  model MLP {
2      input x : tensor(784)
3      output out
4  }
5
6  graph {
7      node fc1 : Linear(in_features=784, out_features=256)
8      node relu1 : ReLU()
9      node drop1 : Dropout(p=0.3)
10     node fc2 : Linear(in_features=256, out_features=128)
11     node relu2 : ReLU()
12     node fc3 : Linear(in_features=128, out_features=10)
13     node out : Softmax(dim=1)
14
15     edge x → fc1
16     edge fc1 → relu1
17     edge relu1 → drop1
18     edge drop1 → fc2
19     edge fc2 → relu2
20     edge relu2 → fc3
21     edge fc3 → out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27     loss = "MSELoss"
28     optimizer = "Adam"
29     lr = 0.001
30     epochs = 5
31 }
32
NNGraphDSL main*
> python main.py -i input/mlp.nng --show-shapes
— Shape inference —
  fc1: (256,)
  relu1: (256,)
  drop1: (256,)
  fc2: (128,)
  relu2: (128,)
  fc3: (10,)
  out: (10,)
Written: output/model.py

```

Figure 6.6: Shape Success Example

6.6 Graphviz Visualization

This inputs nodes, edges, `input_id`, `output_id`, `model_name` and shapes and generates a dot file with colors and stuff to make it beautiful! Not much logic here just making a pretty dot file.

The `dot` tool can then be used to generate an image from the dot file (see Figures 6.7 and

6.8).

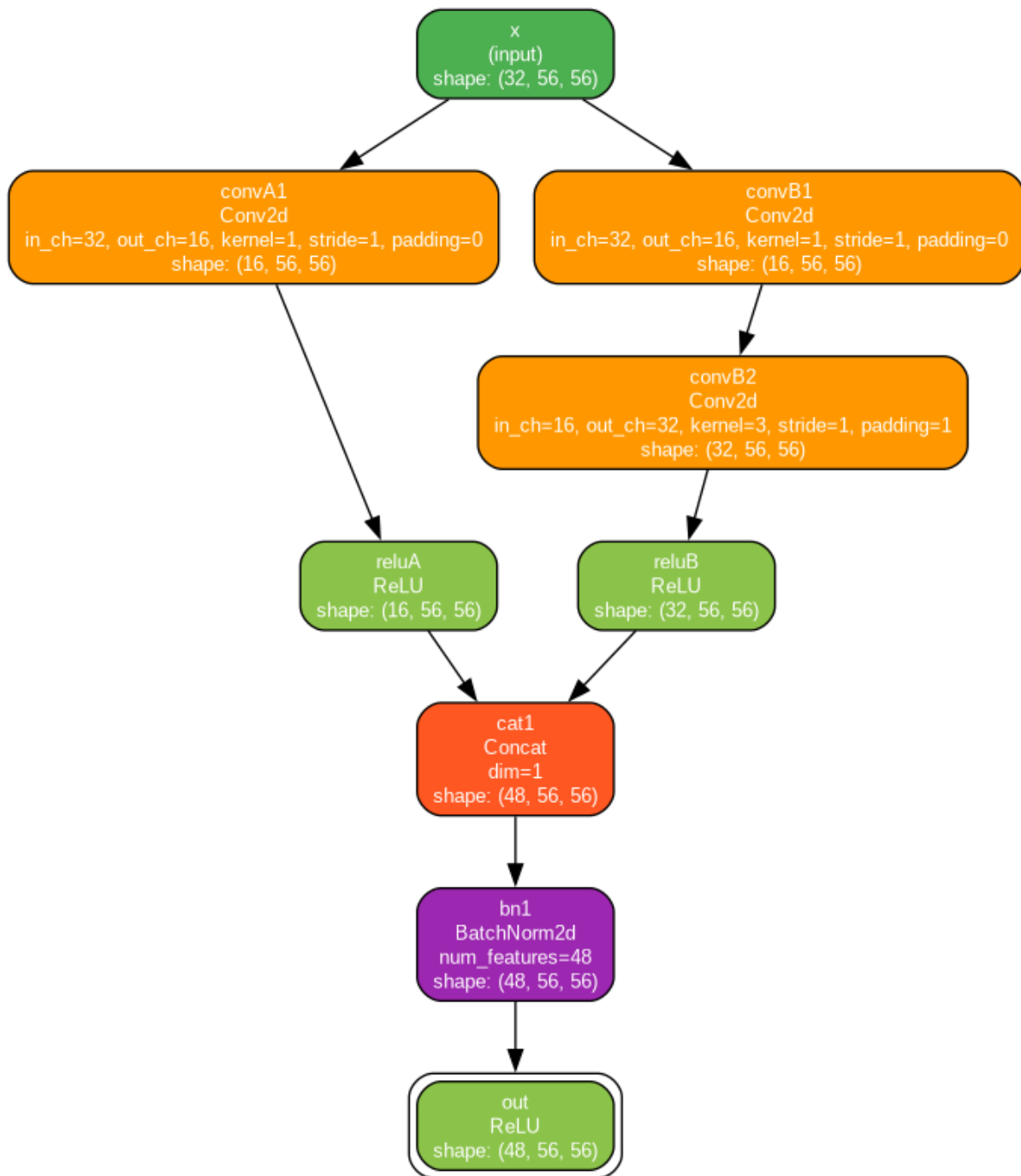


Figure 6.7: Inception Architecture

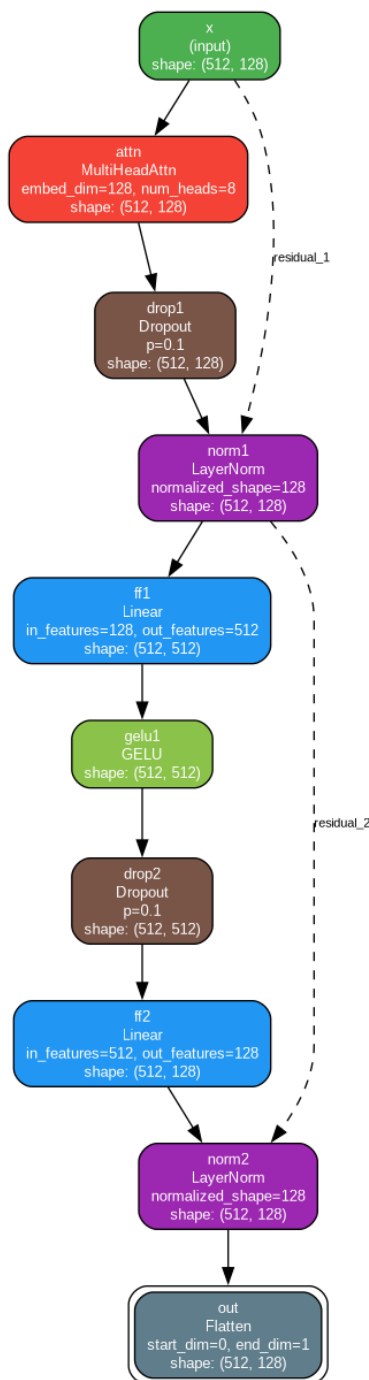


Figure 6.8: Transformer Architecture

6.7 ONNX Export

The `onnx_export.py` module instantiates the generated code and uses `torch.onnx.export` to produce an `.onnx` file with a dynamic batch axis and opset 17.

Conclusion and Future Work

7.1 Conclusion

The implementation of the NNGraph DSL demonstrates the feasibility of representing complex neural network topologies through a domain-specific language and automatically generating their PyTorch implementations.

7.2 Future Work

- **Dynamic Control Flow:** Extending the DSL to support recurrent structures with dynamic lengths or conditional branching (e.g., if-else constructs).
- **Macros and Sub-graphs:** Allowing users to define reusable block macros (like an `InceptionBlock` macro) that can be instantiated multiple times.

References

1. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
2. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
3. NADER Framework: Neural Architecture Design via Representation .
4. Zhang, A. M. *amznotes LaTeX template*. <https://github.com/alexmingzhang/latex-notes-template>